Système de Détection Intelligente de Stationnement

Analyse de l'occupation des places de parking
par vision par ordinateur en temps réel

Vidéo de Démonstration

Lien vers la vidéo du parking :

[ ]

(insérer le lien ici)

Technologie : OpenCV, NumPy, Python 3.13

Méthode : Analyse de variance de texture (sans IA/GPU)

Résolution : 1920 × 1080 pixels

Capacité : 368 emplacements détectés en temps réel

Date : 30 mai 2026

Table des matières

1 Introduction **3**

1.1 Contexte et problématique 3

1.2 Objectifs 3

1.3 Périmètre du rapport 3

2 Description du Système **4**

2.1 Vue d'ensemble 4

2.2 Structure du projet 4

2.3 Dépendances logicielles 5

3 Fonctionnement de l'Algorithme **6**

3.1 Pipeline de traitement 6

3.2 Caractéristiques de texture extraites 6

3.3 Règles de classification 7

3.4 Pourquoi la variance de texture ? 7

3.5 Lissage temporel 7

4 Outils Fournis **8**

4.1 parking_detector.py Pipeline principal 8

4.1.1 Options en ligne de commande 8

4.1.2 Exemples d'utilisation 8

4.2 diagnose.py Outil de diagnostic 9

4.3 annotator.py Annotation interactive des emplacements 9

4.4 Format JSON des emplacements 9

5 Résultats **11**

5.1 Exemple de sortie console 11

5.2 Annotation visuelle 11

6 Résolution des Problèmes **13**

6.1 Erreurs courantes et solutions 13

7 Extensions et Améliorations **15**

7.1 Intégration d'un modèle YOLOv8 15

7.2 Utilisation avec une caméra en direct ou un flux RTSP 15

7.3 Traitement d'une vidéo plus longue 16

8 Conclusion **17**

Chapitre 1

Introduction

1.1 Contexte et problématique

La gestion des espaces de stationnement représente un enjeu majeur dans les environnements urbains modernes. Le manque d'informations en temps réel sur la disponibilité des places entraîne des pertes de temps considérables et une augmentation du trafic inutile.

Ce projet propose un **pipeline de vision par ordinateur** capable de détecter automatiquement l'état d'occupation de chaque emplacement de stationnement à partir d'une séquence vidéo aérienne, *sans nécessiter de modèle d'apprentissage automatique ni de GPU.*

1.2 Objectifs

- Détecter en temps réel si chaque emplacement est **libre**, **occupé** ou en **transition**.
- Afficher des statistiques dynamiques par image (nombre de places libres/occupées et pourcentages).
- Produire une vidéo annotée (MP4) et un aperçu animé (GIF) exploitables directement.
- Offrir des outils d'annotation manuelle et de diagnostic pour faciliter l'adaptation à d'autres parkings.

1.3 Périmètre du rapport

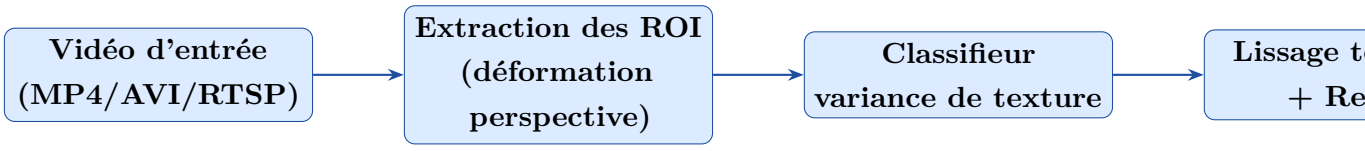
Ce rapport couvre l'architecture du système, le fonctionnement de l'algorithme de classification, la description des outils fournis, les résultats obtenus, ainsi que les pistes d'extension envisageables.

Chapitre 2

Description du Système

2.1 Vue d'ensemble

Le système se compose de quatre modules principaux :



2.2 Structure du projet

Arborescence des fichiers

```
smart-parking-detection/

 parking_detector.py      # Pipeline principal de détection (v6)
 diagnose.py              # Outil de diagnostic des pixels
 annotator.py             # Annotateur interactif d'emplacements
 slots_fixed.json         # Coordonnées corrigées des emplacements
 slots.json               # Coordonnées générées automatiquement
 output/
    parking_detection.mp4   # Vidéo annotée haute résolution
    parking_detection.gif   # Aperçu animé partageable
 README.md
```

TABLE 2.1 – Bibliothèques Python requises

Bibliothèque	**Version minimale**	**Rôle**
`opencv-python-headless`	≥ 4.8	Traitement vidéo et image
`numpy`	≥ 1.24	Calculs numériques
`pillow`	≥ 10.0	Manipulation d'images
`imageio`	≥ 2.31	Export GIF
`scipy`	≥ 1.10	Traitements scientifiques

2.3 Dépendances logicielles

Installation

```
pip install opencv-python-headless numpy pillow imageio
    scipy
```

Chapitre 3

Fonctionnement de l'Algorithme

3.1 Pipeline de traitement

Le traitement de chaque image vidéo suit les étapes suivantes :

1. **Extraction des ROI** chaque polygone d'emplacement est transformé en perspective vers un rectangle fixe de 80×40 pixels.
2. **Calcul des 5 caractéristiques** de texture sur la région normalisée.
3. **Classification** en trois états à partir d'un score $s \in [0, 1]$.
4. **Lissage temporel** pour éviter les changements d'état parasites.
5. **Rendu** des annotations colorées sur l'image et écriture dans les fichiers de sortie.

3.2 Caractéristiques de texture extraites

TABLE 3.1 – Les 5 caractéristiques calculées par emplacement

#	**Caractéristique**	**Description**
1	`std_V`	Écart-type de la brillance (canal V de l'espace HSV)
2	Contraste spatial	Analyse quadrantielle compare les sous-régions entre elles
3	Fraction de zones sombres	Proportion de pixels sous un seuil de luminosité
4	Énergie de gradient Sobel	Détection des contours mesure la netteté des bords
5	Ratio de zones claires	Proportion de pixels très lumineux (reflets de fenêtres, carrosseries)

3.3 Règles de classification

Les seuils de décision sont définis comme suit :

Seuils de classification

$$tat = \{ \ myred\textbf{OCCUPÉ}si\ s \geq 0{,}50\textbf{TRANSITION}si 0{,}28 \leq s < 0{,}50\textbf{LIBRE}si\ s < 0{,}28$$

3.4 Pourquoi la variance de texture ?

Une analyse diagnostique des images aériennes de ce parking a révélé que la brillance moyenne (canal V) et la saturation sont **quasi identiques** entre l'asphalte vide et le toit des véhicules (environ 121122 dans les deux cas). Le seul signal discriminant fiable est la **variance locale de texture** :

- L'asphalte vide est **lisse et uniforme** ($\sigma_V \approx 2028$).
- Les véhicules présentent des **panneaux, vitres et ombres** créant des motifs de luminosité irréguliers ($\sigma_V \approx 4055$).

3.5 Lissage temporel

Un mécanisme de lissage temporel supprime les changements d'état rapides et non fiables. Si la nouvelle confiance est inférieure à 0,30, l'état précédent est maintenu pour éviter les effets de scintillement dans la vidéo de sortie.

Chapitre 4

Outils Fournis

4.1 parking_detector.py Pipeline principal

4.1.1 Options en ligne de commande

Table 4.1 – Arguments CLI du détecteur principal

Option	Défaut	Description
`-video`	`parking_1920_1080.mp4`	Chemin vers la vidéo source
`-slots`	`slots.json`	Fichier de coordonnées des emplacements
`-frames`	`120`	Nombre d'images à traiter
`-out-gif`	`parking_detection.gif`	Nom du fichier GIF de sortie
`-out-mp4`	`parking_detection.mp4`	Nom du fichier MP4 de sortie
`-annotate`	désactivé	Lance l'annotateur interactif
`-debug`	désactivé	Affiche les scores par emplacement (image 1)

4.1.2 Exemples d'utilisation

Listing 4.1 – Exécution standard

```
python parking_detector.py --video parking_1920_1080.mp4 \
    --slots slots_fixed.json --frames 120
```

Listing 4.2 – Exécution avec débogage

```
python parking_detector.py --video parking_1920_1080.mp4 \
    --slots slots_fixed.json --frames 120 --debug
```

4.2 diagnose.py Outil de diagnostic

Cet outil analyse les statistiques de pixels de votre vidéo afin de comprendre quels signaux (brillance, saturation, variance de texture) sont exploitables pour votre séquence spécifique.

Sorties générées :

- Statistiques HSV moyennes par emplacement (H, S, V, std_V, densité de contours).
- Positions des bandes de voie détectées.
- Comparaison emplacements vs voies.
- Image de débogage `debug_frame.png` (contours jaunes = emplacements, cyan = voies).

```
python diagnose.py --video parking_1920_1080.mp4 \
    --slots slots_fixed.json
```

4.3 annotator.py Annotation interactive des emplacements

Un outil graphique permettant de dessiner manuellement les polygones d'emplacements ou de corriger des annotations existantes.

TABLE 4.2 – Raccourcis clavier de l'annotateur

Action	**Effet**
Clic gauche	Placer un coin (4 par emplacement)
ESPACE	Valider l'emplacement (après 4 points)
CTRL+Z	Annuler le dernier point ou emplacement
S	Sauvegarder sans quitter
Q / Échap	Sauvegarder et quitter
Molette de défilement	Zoom avant/arrière
Clic molette + glisser	Panoramique

La sauvegarde automatique intervient tous les 10 emplacements confirmés. Le résultat est écrit dans `slots_new.json` par défaut.

4.4 Format JSON des emplacements

Chaque emplacement est défini par un identifiant unique et un polygone de 4 points (sens horaire depuis le coin supérieur gauche) :

Listing 4.3 – Exemple de fichier slots.json

```
[
  {
    "id": 1,
    "polygon": [[88, 103], [163, 103], [163, 130], [88,
       130]]
  },
  {
    "id": 2,
    "polygon": [[88, 130], [163, 130], [163, 163], [88,
       163]]
  }
]
```

Chapitre 5

Résultats

5.1 Exemple de sortie console

L'exécution sur 120 images d'une vidéo de 849 images au total produit la sortie suivante :

Sortie console

```
Loaded 368 slots from slots_fixed.json
Video: 1920x1080 @ 30.0 fps (849 frames total)
Processing up to 120 frames (every 2nd)...

Frame 20 | Occ: 87 Emp:241 Trans: 40
Frame 40 | Occ: 89 Emp:239 Trans: 40
Frame 60 | Occ: 91 Emp:238 Trans: 39
...

MP4 -> output\parking_detection.mp4
GIF -> output\parking_detection.gif
Done. Processed 120 frames, 368 slots.
```

5.2 Annotation visuelle

Chaque image de la vidéo de sortie est annotée en temps réel selon le code couleur suivant :

Un **panneau de statistiques** en haut à gauche de chaque image affiche en temps réel les comptages et pourcentages d'occupation.

TABLE 5.1 – Code couleur des emplacements détectés

Couleur	**Symbole**	**Signification**
Vert		Emplacement libre
Rouge		Emplacement occupé
Jaune		Transition (arrivée ou départ)

Chapitre 6

Résolution des Problèmes

6.1 Erreurs courantes et solutions

AttributeError: module 'cv2' has no attribute 'VideoCapture'

Plusieurs paquets OpenCV incompatibles sont installés simultanément. Résolution :

```
pip uninstall opencv-python opencv-python-headless \
    opencv-contrib-python -y
pip install opencv-python-headless
```

Tous les emplacements affichés comme OCCUPÉS

1. Exécuter `diagnose.py` et vérifier la ligne `Avg slot vs lane V diff`.
2. Si le différentiel est faible, la référence aux voies n'est pas exploitable le classifieur de texture est la bonne approche.
3. Lancer avec `-debug` et ajuster les seuils `OCCUPIED_THRESH` et `EMPTY_THRESH` dans `parking_detector.py`.

Emplacements couvrant les voies de circulation

Les coordonnées de `slots.json` ne correspondent pas à la vidéo. Utiliser `slots_fixed.json` ou ré-exécuter le script de cartographie avec une nouvelle image `frame0.png`.

ERROR: slots file not found

S'assurer que `slots_fixed.json` se trouve dans le même dossier que `parking_detector.py`, et non dans le sous-dossier `output/`.

Chapitre 7

Extensions et Améliorations

7.1 Intégration d'un modèle YOLOv8

Pour une précision de détection supérieure, il est possible de remplacer le classifieur de texture par un modèle de deep learning :

Listing 7.1 – Intégration YOLOv8 (classe 2 = voiture dans COCO)

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")

def classify(self, roi, debug=False):
    results = model(roi, verbose=False)
    for box in results[0].boxes:
        if int(box.cls) == 2:  # classe 2 = voiture (COCO)
            return "OCCUPIED", float(box.conf), 1.0
    return "EMPTY", 0.95, 0.0
```

7.2 Utilisation avec une caméra en direct ou un flux RTSP

```
# Caméra locale (index 0)
python parking_detector.py --video 0

# Flux IP (RTSP)
python parking_detector.py --video rtsp://192.168.1.100/
   stream
```

7.3 Traitement d'une vidéo plus longue

```
python parking_detector.py --video parking_1920_1080.mp4 \
    --slots slots_fixed.json --frames 500
```

Chapitre 8

Conclusion

Ce projet démontre qu'une détection robuste de l'occupation des places de stationnement est réalisable **sans modèle d'apprentissage automatique ni matériel GPU**, à condition d'identifier et d'exploiter le bon signal discriminant ici la variance de texture locale dans l'espace HSV.

Le pipeline traite en temps réel 368 emplacements sur une vidéo 1920Œ1080 pixels, génère des sorties annotées directement exploitables, et propose des outils complémentaires (diagnostic, annotation manuelle) pour faciliter l'adaptation à d'autres configurations de parkings.

Les perspectives d'amélioration incluent l'intégration optionnelle d'un modèle YOLOv8 pour les cas difficiles, la compatibilité avec des flux RTSP pour le déploiement en conditions réelles, et le développement d'un tableau de bord web pour la visualisation centralisée des données d'occupation.

Rapport généré à partir du projet *Smart Parking Detection System*.
Construit avec OpenCV, NumPy et Python.
Classifieur calibré sur analyse diagnostique de séquences aériennes 1920Œ1080.